

Unit 2 – Requirements Engineering

Outcomes & Introduction

Unit 2 Outcomes

- Appreciate the Importance of Requirements Engineering Phase
- Differentiate User and System Requirements
- Understand the difference between Functional and Non-Functional Requirements



Unit 2 Outcomes

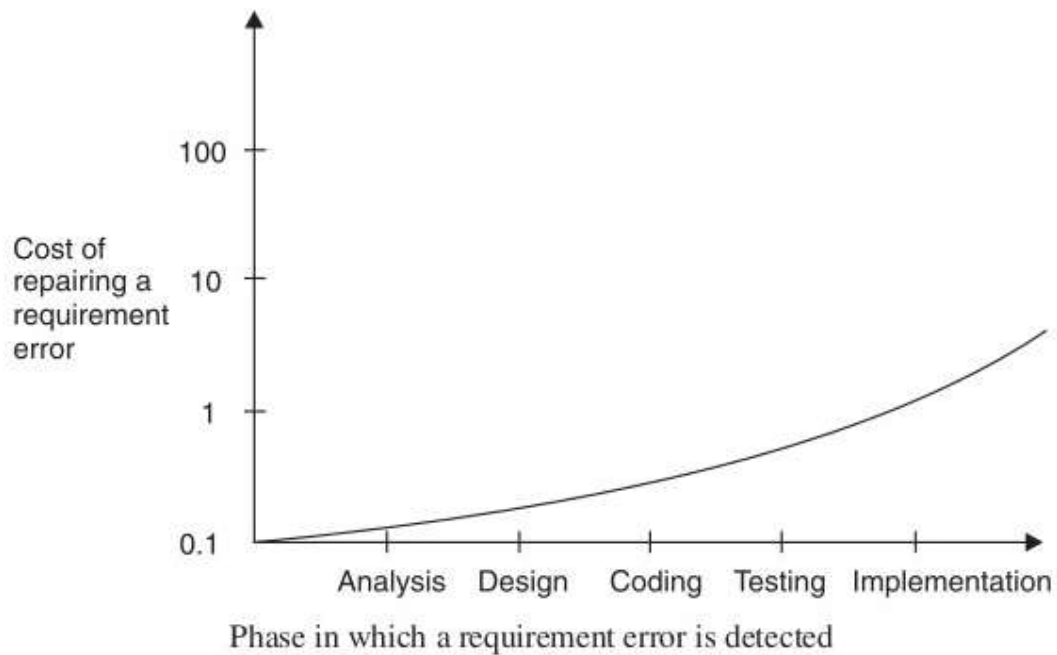
- Examine the **Structure** of a Software Requirements (**SRS**) document
- Recognize the different **activities** in Requirements Engineering like **Elicitation**, **Analysis**, **Validation** and **Management**
- **Tools** for Gathering Requirements for example **Decision Tables**, **Decision Trees**, **Data Flow diagrams**

Importance of Requirements Engineering Phase

- Requirements are the **MOST** important element required to construct a Software
- **Essential** before commencing software development
- **Clear** and **well specified** to obtain the desired functions from the software
- **Cost** of Requirement **Changes** is extremely high as time passes by



Cost of Correcting a Requirement Error



Unit 2 – Requirements Engineering

Types of Requirements

Introduction

- Descriptions of the **services** that a system should provide and the **constraints** on its operation
- Reflect the **needs** of customers for a system
- Process of **finding out, analyzing, documenting** and checking these services and **constraints** is called **requirements engineering**

User Requirements

- **User Requirements** Specify the high-level abstract requirements
- **System Requirements** Detailed description of what the system needs to do
- Defines **services** and Operating **Constraints**
- **Broad** statements or **detailed, precise** descriptions of system functionality
- Language of the User



System Requirements

- Detailed descriptions of the software system's, functions, services, and operational constraints
- System Requirements Document also known as Functional Specification
- Define exactly what is to be implemented
- Forms basis of Contract between developers and the contracting company

Terminology - Stakeholders

- Anyone who is affected by the system in some way and so anyone who has a legitimate interest in it
 - Client Managers
 - End Users
 - Software Developers
 - System Designers/Architects



MHC-PMS – User Requirements

The system should generate **monthly management reports** showing the **cost** of the **drugs** prescribed during that month.

MHC-PMS – System Requirements

- On the last working day of the month, a **summary** of the **drugs prescribed**, their **costs** shall be generated.
- The system shall generate the report for printing after **17:30** on the **last working day**.
- If drugs are available in different dose units (eg. 10mg, 20mg etc.) **separate reports** shall be generated for each dose unit.
- **Access** to drug cost report shall be **restricted** to authorized users as listed on a access control list.



MHC-PMS - Stakeholders

- **Patients** whose information is recorded in the system and relatives of these patients.
- **Doctors** who are responsible for assessing and treating patients.
- **Nurses** who coordinate the consultations with doctors and administer some treatments.
- **Medical receptionists** who manage patients' appointments.
- **IT staff/Medical Records** who are responsible for installing and maintaining the system records.

Unit 2 – Requirements Engineering

Functional & Non-Functional Requirements



Functional Requirements

- **Basic features** that the system should offer to a End User
- All these functionalities need to be necessarily incorporated into the system as a part of the **contract**
- **Clearly** describe each functionality that the system would **support**

Examples of Functional Requirements

- **Authentication** of user whenever they log into the system
- **Verification email** is sent to user whenever they register for the first time on some software system.
- Information to be captured from the user
- **Search** for Trains between two stations
- Support Payments using **BHIM-UPI**



Non-Functional Requirements

- Quality **constraints** that the Software Application must satisfy
- Address aspects concerning **external interfaces**, user interfaces, maintainability, **portability**, usability, maximum number of concurrent users, timing, and **throughput** etc
- **Extremely important and failure may lead to non-acceptance of the software**

Examples of Non-Functional Requirements

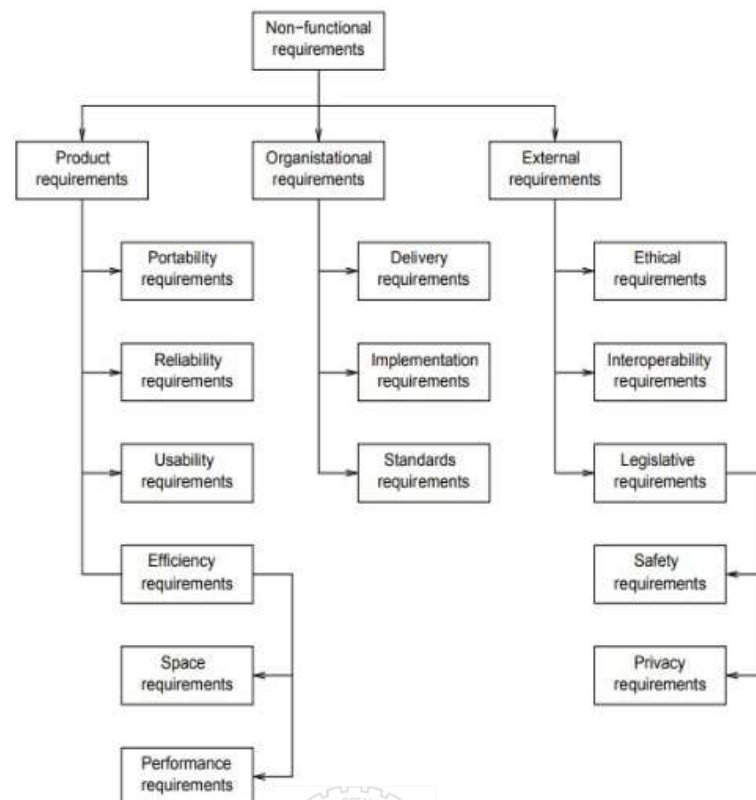
- The client expect **10,000** credit card statements to be generated/processed **per minute**
- Concurrently **25000** students shall be able to check their results **per second**
- Software shall be developed using **Open Source tools** only
- Software development team members shall be **citizens of India** Only



Unit 2 – Requirements Engineering

Non-Functional Requirements Types and Metrics

Types of Non-Functional Requirements



Metrics for Non-Functional Requirements

Speed	Processed transactions/second : User/event response time Screen refresh time
Size	Mbytes
Ease of use	Training time : Number of help frames
Reliability	Mean time to failure : Probability of unavailability Rate of failure occurrence : Availability
Robustness	Time to restart after failure : Percentage of events causing failure Probability of data corruption on failure
Portability	Percentage of target dependent statements Number of target systems

Unit 2 – Requirements Engineering

Activities in Requirements Engineering



Activities in Requirements Engineering

- **Feasibility Study**
- Requirement's **Elicitation** and **Analysis**
- Requirement's **Specification**
- Requirements **Verification** and **Validation**
- Requirements **Management**

Feasibility Study

- Customer's needs within the existing **budget** and **technology** conditions
- Feasibility report is generated that mainly determines whether the system fulfills the **objectives of the organization**
- Further detailed requirement analysis can be conducted or not



Requirement's Elicitation and Analysis

- Gather information about the **project domain** and **requirements**
- Customers, business manuals, the existing software, **stakeholders** of the project
- Interviews, brainstorming, task analysis, **prototyping**

Requirement's Specification

- Produces formal software requirement **models**
- **Functional** as well as the **non-functional** requirements
- **ER diagrams**, data flow diagrams (DFD's), data dictionaries



Requirements Verification and Validation

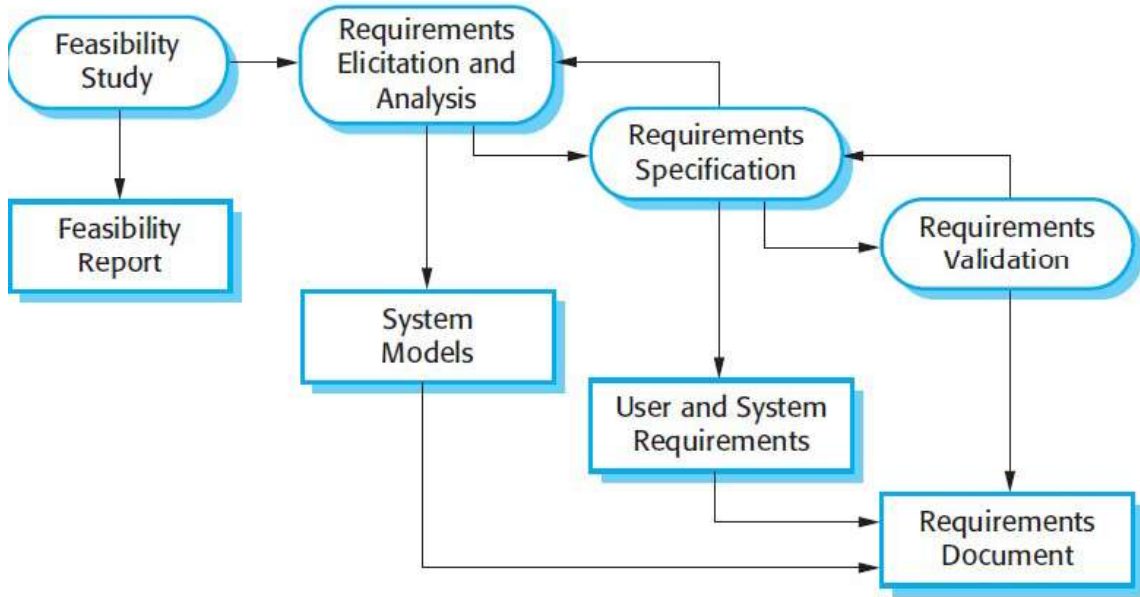
- **Verification** refers to the set of tasks that ensures that the software correctly **implements a specific function**
- **Validation** refers to a different set of tasks that ensures that the software that has been built is **traceable to customer requirements**
- **Reviews, and test cases**

Requirements Management

- **Analyzing, documenting, tracking, prioritizing** and agreeing on the requirement and controlling the communication to relevant stakeholders
- **Takes care of the changing nature of requirements**



Requirements Engineering Process



Unit 2 – Requirements Engineering

Feasibility Study

Feasibility Study

- Takes place in the **early stage** of RE process
- Questions such as whether the **objectives** of the organization
- Implemented within **budget** and by the latest **technology**
- Decision made to **implement or abandon** the project

Types of Feasibility Study

- **Technical** Feasibility
- **Operational** Feasibility
- **Legal** Feasibility
- **Schedule** Feasibility



Activities in a Feasibility Study

- Introduction
- Summary & Recommendations
- Alternatives
- System Description
- Cost-Benefit Analysis
- Evaluation of Risks
- Other Project-Specific Topics

Unit 2 – Requirements Engineering

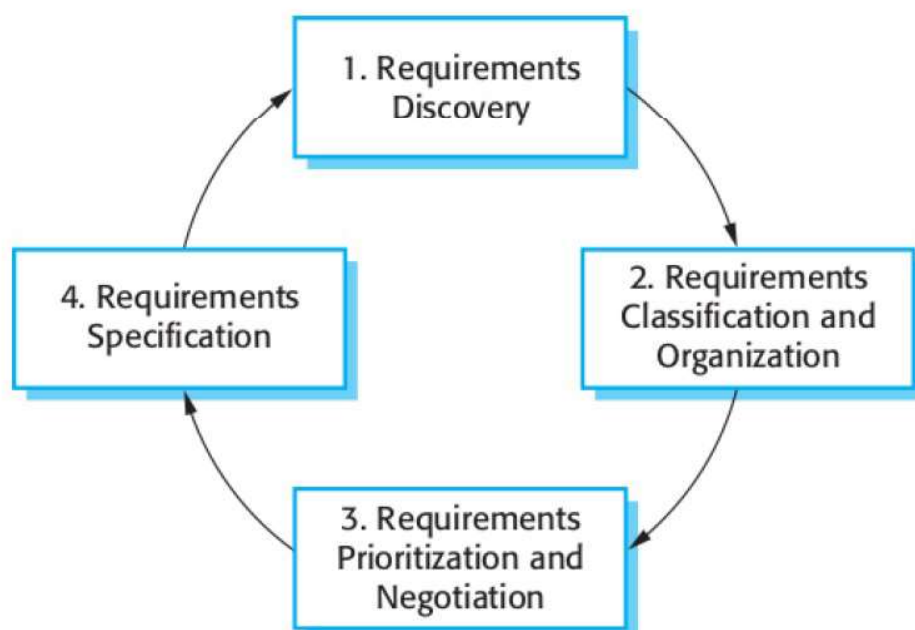
Requirements Elicitation and Analysis



Introduction

- Understand the **work/requirements** of stakeholders
- Find out about the **application domain**, work activities, the **services** and **system features**
- The required **performance** of the system, hardware constraints

Requirements Elicitation Process



Requirements discovery/elicitation

- The process of **gathering information** about the **required** and **existing systems** and figuring out the **user** and **system requirements** from this information.
- This stage involves interacting with **stakeholders** to discover their requirements. Domain requirements are also discovered at this stage.

Requirements classification and organization

- This activity takes the **unstructured** collection of requirements, groups related requirements and organizes them into **coherent clusters**
- Generally by Functional Modules



Requirements prioritization and negotiations

- Generally when many stakeholders are involved there are **conflicts** in requirements.
- This activity is concerned with **prioritizing** requirements and resolving requirements conflicts.

Requirements documentation

- Requirements are documented and an early draft of the **software requirements document** may be produced at this stage.
- It is important that you use **simple language** and **diagrams** to describe the requirements.
- This makes it possible for stakeholders to **understand** and **comment** on these requirements.



Unit 2 – Requirements Engineering

Requirements Elicitation Techniques

Introduction

- Various **techniques** can be used to gather requirements.
- They generally involve **meeting** with **stakeholders**.
- This information can be further supplemented with knowledge of **existing systems** and their **usage** and **documentation** about them



Techniques for Eliciting Requirements

- ❑ Interviewing
- ❑ Observation or Ethnography
- ❑ Scenarios
- ❑ Use cases
- ❑ Survey
- ❑ Existing manuals and documentation
- ❑ Group Discussion
- ❑ Prototyping
- ❑ Brainstorming
- ❑ Questionnaires

Sample Questions

- ❖ What is the **goal** of developing this system? Who are the users and which goal is the most important?
- ❖ Will this system help your office to be more efficient? How?
- ❖ This project should be developed as a **desktop application/Database/Mobile App/Web service/Client-server** type?
- ❖ What is **time limit** to make this project functional? (days/months/years)
- ❖ How many **IT staff member** can be involved? (Depending on budget)



Unit 2 – Requirements Engineering

Barriers to Requirements Elicitation

Introduction

- Getting **precise**, implementable software requirements is possibly the **most challenging** Software Engineering task
- Some **Reasons are explored** in the following slides



Possible Reasons [1/2]

- Stakeholders often **don't know** what they want from a computer system.
- They may find it difficult to **articulate** what they want the system to do.
- They may make **unrealistic demands** because they don't know what is and isn't feasible.
- Software normally serves a **variety of users**; each user focuses on different issues associated with the overall problem addressed by the software.
- Each has a **separate view** of the problem.

Possible Reasons... [2/2]

- A lack of **communication** between the users and analysts/developers can cause issues while gathering requirements.
- Even characteristics like the analysts **interpersonal** skills and **personality** can cause issues.
- **Unwillingness** of some users to provide requirements.
- For example, they may feel **uncomfortable** with a new system trying to **replace** part of their jobs in the organization.



Unit 2 – Requirements Engineering

Requirements Specification

Introduction

- Means Documenting **User** and **System** Requirements
- Goal is document requirements **clearly, completely, and unambiguously**
- **User Requirements** usually in **Natural Language** and some **Diagrams**
- **System Requirements Graphical Notations** and **Mathematical symbols**



System Requirements

- Expanded versions of user requirements, that the software engineers use for **system design**
- Detailed and explains how the system should provide the user requirements
- Used for **contract** so needs to be complete and **detailed** specification of the whole system

Natural Language Specification

- Natural language is **expressive**, **intuitive** and **universal**, but it could also be **vague** and **ambiguous**
- **For example:** Software based system to measure the blood sugar and deliver insulin
- Measure the blood sugar and deliver insulin, if required, **every 10 minutes**. (Changes in blood sugar are relatively slow, so more frequent measurement is unnecessary; less frequent measurement could lead to unnecessarily high sugar levels.)



Structured Specification

Insulin Pump/Control Software/SRS/3.3.2

Function	Compute insulin dose: Safe sugar level.
Description	Computes the dose of insulin to be delivered when the current measured sugar level is in the safe zone between 3 and 7 units.
Inputs	Current sugar reading (r2), the previous two readings (r0 and r1).
Source	Current sugar reading from sensor. Other readings from memory.
Outputs	CompDose—the dose in insulin to be delivered.
Destination	Main control loop.
Action:	CompDose is zero if the sugar level is stable or falling or if the level is increasing but the rate of increase is decreasing. If the level is increasing and the rate of increase is increasing, then CompDose is computed by dividing the difference between the current sugar level and the previous level by 4 and rounding the result. If the result, is rounded to zero then CompDose is set to the minimum dose that can be delivered. (see Figure 4.14)
Requires	Two previous readings so that the rate of change of sugar level can be computed.
Precondition	The insulin reservoir contains at least the maximum allowed single dose of insulin.
Postcondition	r0 is replaced by r1 then r1 is replaced by r2.
Side effects	None.

Unit 2 – Requirements Engineering

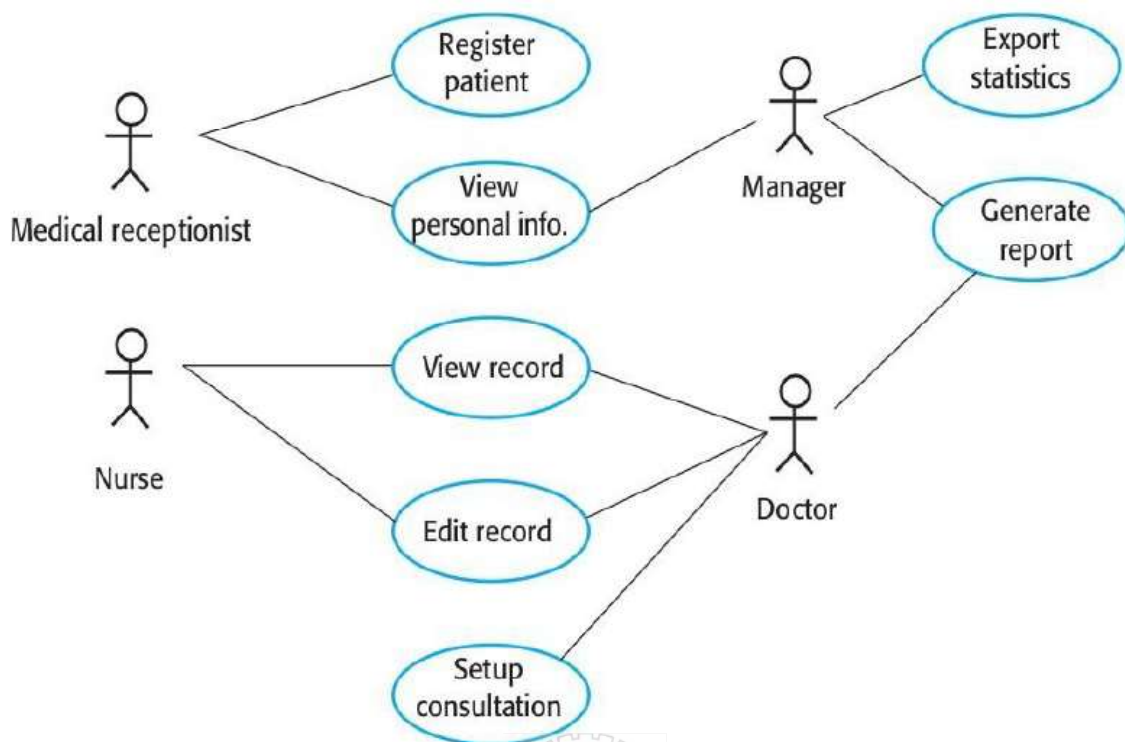
Requirements Specification using Graphical Notation & Mathematical Specifications



Introduction

- Graphical models, supplemented by text annotations, are used to define the functional requirements for the system
- UML (unified modeling language) use case and sequence diagrams are commonly used
- Use cases are a way of describing interactions between users and a system using a graphical model and structured text

Sample Use Case



Mathematical Specifications

- Based on mathematical concepts such as **finite-state machines** or sets
- Help reduce the **ambiguity** in a requirements document
- But most **customers** don't understand a formal specification
- Used in highly technical **safety critical applications** where errors in requirements are life threatening

Unit 2 – Requirements Engineering

Software Requirements Specification (SRS)



SRS - Introduction

- Systematically **organize** the requirements in the form document (SRS)
- **Official statement** of what is required of the **software**
- Definition of **user requirements** and a specification of the **system requirements**
- Most **important document** and is the **toughest** to write

SRS – Utility (1/2)

- Contains **functional** requirements, **non-functional** requirements and **goals** of implementation
- **Customers/Users/Clients** specify the requirements and read them to check that they meet their needs
- **Bid** for the system and to **plan** the system development process



SRS – Utility (2/2)

- **System engineers** use the requirements to understand the system that must be developed
- **System test engineers** use the requirements document to develop **validation tests** for the system
- **System maintenance engineers** use the requirements document to understand the system and the relationships between its parts

Unit 2 – Requirements Engineering

Software Requirements Specification (SRS) Good and Bad Practices



SRS – Good Practices (1/3)

Correctness: covers all the requirements that are actually expected from the system.

Completeness: covering all the functional and non-functional requirements properly.

Consistency: no conflicts between any set of requirements. Examples of conflict include differences in terminologies used at separate places, logical conflicts like time period of report generation, etc.

SRS – Good Practices (2/3)

Concise : short and concise document.

Unambiguous : requirements stated have only one interpretation.

Structured and easy to understand by the customers

Black-box view : not include any design or implementation details.

Verifiable : quantifiable measure the extent to which every requirement is met by the system.



SRS – Good Practices (3/3)

Modifiable/Maintainable : capable of easily accepting changes to the system to some extent.

Traceable : trace a requirement to **design** component and then to **code segment** in the program. Similarly, one should be able to trace a requirement to the corresponding **test cases**.

Testable : generate test cases and test plans from the document.

SRS – Bad Practices

Over-specification : too detailed, it would be difficult to understand and use

Forward references : First all terms and prerequisites should be made clear and only then should requirements be specified

Wishful thinking : should be implementable.

Noise : Unnecessary details



Unit 2 – Requirements Engineering

SRS

IEEE Format

SRS – IEEE Format (1/4)

1. Introduction

1.1 Purpose

1.2 Project Scope

1.3 Definitions, acronyms and abbreviations

1.4 References

1.5 Overview



SRS – IEEE Format (2/4)

2 Overall Description

- 2.1 Product perspective
- 2.2 Product features
- 2.3 User classes
- 2.4 Operating environment
- 2.5 Design and implementation constraints
- 2.6 User documentation
- 2.7 Assumptions and Dependencies

SRS – IEEE Format (3/4)

3 Specific Requirements (Features)

- 3.1 Functional Requirement 1
- 3.2 Functional Requirement 2 ...

4 External Interface Requirements

- 4.1 User interfaces
- 4.2 Hardware interfaces
- 4.3 Software interfaces
- 4.4 Communication interfaces



SRS – IEEE Format (4/4)

5 Other non-functional Requirements

5.1 Performance Requirements

5.2 Safety Requirements

5.3 Security Requirements

5.4 Software Quality Attributes

6 Other Requirements

7 Appendix A: Glossary

8 Appendix B: Analysis Models

9 Appendix C: To Be Determined List

Unit 2 – Requirements Engineering

SRS – Case Study Part 1 (1 - Introduction)



SRS

1. Introduction

1.1 Purpose

1.2 Project Scope

1.3 Definitions, acronyms and abbreviations

1.4 References

1.5 Overview

1. Introduction (1/4)

1.1 Purpose

The purpose of this SRS document is to describe the proposed online **flight management** system enabling...

- Passengers to manage their travel
- Airline to manage flights and passengers travel



1. Introduction (2/4)

1.2 Project Scope

The purpose of the **online flight management system** is to ease flight management and to create a convenient and easy-to-use application for passengers, trying to buy airline tickets. The system is based on a relational database with its **flight management** and **reservation** functions.

We will have a **distributed database** supporting management of thousands of flights by various airline companies.

1. Introduction (3/4)

1.3 Definitions, Acronyms and Abbreviations

This document uses the following conventions.

DB	Database
DDB	Distributed Database
ER	Entity Relationship



1. Introduction (4/4)

1.4 References

- ❖ <https://krazytech.com/projects>
- ❖ Fundamentals of database systems by Ramez Elmarsi and Shamkant Navathe

1.5 Overview

We provide the overall description of the project in Section 2. The specific requirements are listed in Section 3. The Sections 4 and 5 give the external interface requirements and non-functional requirements respectively.

Unit 2 – Requirements Engineering

SRS – Case Study Part 2 (2 Overall Description)



SRS – 2 Overall Description

2.1 Product perspective

2.2 Product features

2.3 User classes

2.4 Operating environment

2.5 Design and implementation constraints

2.6 User documentation

2.7 Assumptions and Dependencies

2. Overall Description

2.1 Product Perspective

The software application is being designed to **replace** the **legacy application**. The legacy software was unsuited to serve the customers and internal users in the context of latest **technological developments** facilitated by the Internet accessibility.

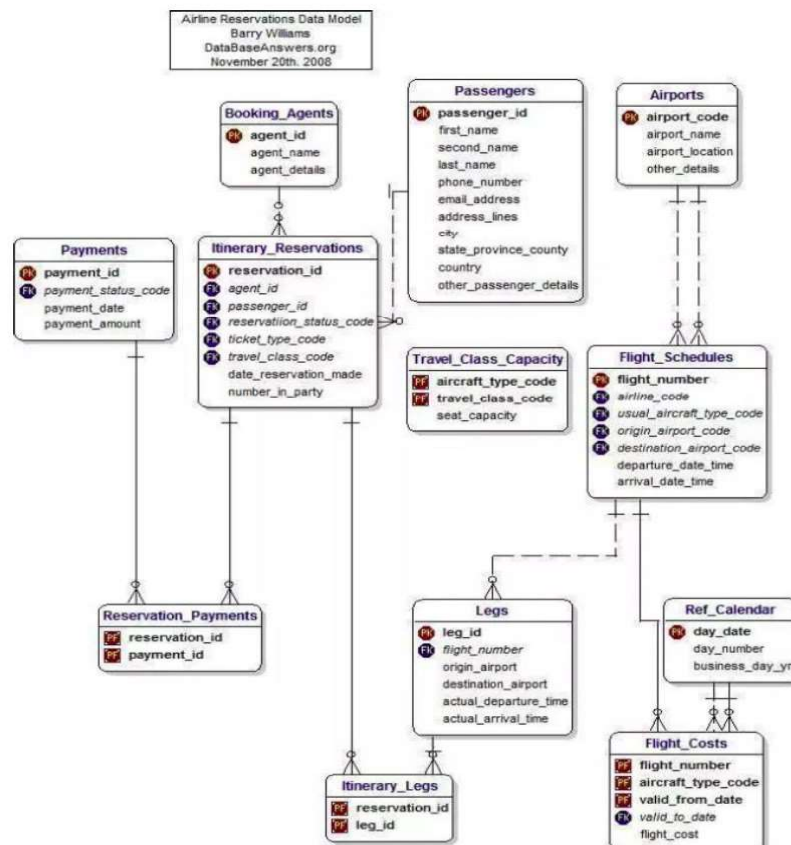
The earlier system was restricted to internal staff and travel agents. The new software is meant to provide **access to customers** to manage their travel requirements by themselves.



2. Overall Description

2.2 Product Features

The major features of flight management system are as shown in the following slide.



The diagram shows the layout of airline database system



2. Overall Description

2.3 User Classes

Users of the system should be able to **retrieve flight information** between **two given cities** with the given date/time of travel from the database. A route from city **A** to city **B** is a sequence of connecting flights from **A** to **B** such that: a) there are at most two connecting stops, excluding the starting city and destination city of the trip, b) the connecting time is between one to two hours.

The system will support two types of user privileges, **Customer**, and **Employee**. Customers will have access to **customer functions**, and the employees will have access to both **customer** and **flight management functions**.

2. Overall Description

Customer Functions

- Make a new reservation
- One-way
- Round-Trip
- Multi-city
- Flexible Date/time
- Confirmation
- Cancel an existing reservation
- View their itinerary



2. Overall Description

Employee Functions

- ▣ Get all customers who have seats reserved on a given flight.
- ▣ Get all flights for a given airport.
- ▣ View flight schedule.
- ▣ Get all flights whose arrival and departure times are on time/delayed.
- ▣ Calculate total sales for a given flight.

2. Overall Description

Employee - Administrative Functions

- ▣ Add/Delete a flight
- ▣ Add a new airport
- ▣ Update fare for flights.
- ▣ Add a new flight leg instance
- ▣ Update departure/arrival times for flight leg instances.



2. Overall Description

2.4 Operating Environment

- Distributed Database
 - Client/Server System
 - Server Operating System: Windows.
 - Database: SQL+ database
 - Platform: vb.net/Java/PHP
- The **client-side components** of the software system must operate within common web-browser environments using Secure Sockets Layer (SSL) / Transport Layer Security (TLS) cryptographic protocols at a minimum **encryption level of 128 bits**.

2. Overall Description

2.5 Design and Implementation Constraints

- Flight dates and hours should be displayed according to the **city of departure and destinations' time zones** and the **daylight saving time** settings for each country should be considered.
- Provision for support of Languages other than English in the future for user communication



2. Overall Description

2.6 User Documentation

A range of short document types (e.g., **guidelines, tutorials, frequently asked questions**) in Hypertext Markup Language (HTML) and/or Portable Document Format (PDF) format must describe the **use** of the software system for customers and staff.

2. Overall Description

2.7 Assumptions and Dependencies

- ❑ Users have an internet access with standard browsers and can do online payments using Visa, MasterCard and Net Banking only
- ❑ Supports Android based Mobile Devices only
- ❑ Shall be using third party component for payment processing



Unit 2 – Requirements Engineering

SRS – Case Study Part 3 **(3 Specific Requirements)**

SRS – 3 Specific Requirements

3 Specific Requirements (Features)

3.1 Functional Requirement 1

3.2 Functional Requirement 2 ...



3 Specific Requirements

Description and Priority

- The airline reservation system maintains information on flights, classes of seats, personal preferences, prices, and bookings. Project has a high priority because it is very difficult to travel across countries without prior reservations.

Stimulus/Response Sequences

- **Search** for Airline Flights for two Travel cities
- **Displays** a detailed list of available flights and make a “Reservation” or Book a ticket on a particular flight.
- **Cancel** an existing Reservation.

3 Specific Requirements

Distributed Database

- Distributed database implies that a single application should be able to operate transparently on data that is spread across a variety of different databases and connected by a communication network



3 Specific Requirements

Client/Server System

- The term client/server refers primarily to an architecture or logical division of responsibilities, the client is the application (also known as the front-end), and the server is the DBMS (also known as the back-end).

A client/server system is a distributed system in which,

- Some sites are client sites and others are server sites.
- All the data resides at the server sites.
- All applications execute at the client sites

Unit 2 – Requirements Engineering

SRS – Case Study Part 4 (4 External Interfaces)



SRS – 4 External Interfaces

4 External Interface Requirements

4.1 User interfaces

4.2 Hardware interfaces

4.3 Software interfaces

4.4 Communication interfaces

4 External Interfaces

4.1 User Interfaces

- The user interface for the software shall be compatible to any browser such as Internet Explorer, Mozilla by which user can access to the system. The user interface shall be implemented using any tool or software package Java Applet, MS Front Page, EJB .

4.2 Hardware Interfaces

- Since the application must run over the internet, all the hardware shall require to connect internet will be hardware interface for the system. As for e.g. Modem, WAN – LAN, Ethernet Cross-Cable.



4 External Interfaces

4.3 Software Interfaces

Software used	Description
Operating system	We have chosen Windows operating system for its best support and user-friendliness.
Database	To save the flight records, passengers records we have chosen SQL+ database.
VB.Net	To implement the project we have chosen Vb.Net language for its more interactive support.

4.4 Communication Interfaces

The system shall use the HTTP protocol for communication over the internet and for the intranet communication will be through TCP/IP protocol suite.

Unit 2 – Requirements Engineering

SRS – Case Study Part 5 (5 Non-Functional Requirements)



SRS – 5 Non-Functional Requirements

5 Other non-functional Requirements

5.1 Performance Requirements

5.2 Safety Requirements

5.3 Security Requirements

5.4 Software Quality Attributes

5 Non-Functional Requirements

5.1 Performance Requirements

The flight management system should respond back within 10ms for any request

5.2 Safety Requirements

If there is extensive damage to a wide portion of the database due to **catastrophic failure**, such as a disk crash, the recovery method restores a past copy of the database that was backed up to archival storage (typically tape) and reconstructs a more **current state** by reapplying or redoing the operations of committed transactions from the backed up log, up to the time of failure.



5 Non-Functional Requirements

5.3 Security Requirements

Data Transfer

- ❖ The system shall use **secure sockets** in all transactions that include any confidential customer information.
- ❖ The system shall automatically **log out** all customers after a period of inactivity.
- ❖ The system shall not leave any **cookies** on the customer's computer containing the user's **password**.
- ❖ The system shall not leave any cookies on the customer's computer containing any of the **user's confidential information**

5 Non-Functional Requirements

Data Storage

- ❑ The customer's web browser shall never display a customer's **password**. It shall always be echoed with special characters representing typed characters.
- ❑ The customer's web browser shall never display a **customer's credit card** number after retrieving from the database. It shall always be shown with just the last 4 digits of the credit card number.
- ❑ The system's back-end servers shall only be accessible to **authenticated** administrators.
- ❑ The system's back-end databases shall be **encrypted**.



5 Non-Functional Requirements

5.4 Software Quality Attributes

Availability:

Because this application is critical to business communication, we will have a goal of four nines(99.99%) availability

Maintainability:

The application should use continuous integration so that features and bug fixes can be deployed quickly without downtime.

Usability :

The interface should be easy to learn without a tutorial and allow users to accomplish their goals without errors

Unit 2 – Requirements Engineering

Requirements Validation



Need for Validation

- ❑ Process of checking that requirements define the system that the **customer really wants**
- ❑ Very important because errors in a requirements document can lead to extensive **rework costs**
- ❑ Different types of checks are carried out on the requirements in the requirements document

Requirements Validation Checks

- ❑ **Validity checks** : Alternate or Additional Functions required to satisfy requirements
- ❑ **Consistency checks** : Requirements in the document should not conflict
- ❑ **Completeness checks** : These check that requirements document contains all the requirements that define all functions and constraints given by the system users



Requirements Validation Checks

- **Realism checks** : These check if the requirements can be implemented within the available budget and technology constraints
- **Verifiability** : This means that you should be able to write a set of tests that can demonstrate that the delivered system meets each specified requirement.

Requirements Validation Techniques

- **Requirements Reviews** : The requirements are analyzed systematically by a team of reviewers who check for errors and inconsistencies.
- **Test-case generation** : Tests are written for requirements as a part of the validation process. These tests often reveal requirements problems. Developing such tests even before writing any code is an important part of test-driven development
- **Prototyping**



Unit 2 – Requirements Engineering

Requirements Management

Need for Requirements Management (1/2)

- ❑ Requirements keep on **changing**
- ❑ During the software development process, the **stakeholder's understanding** of the problem evolves
- ❑ Requirements management is the process of **managing changing requirements** during the requirements engineering process and **system development**



Need for Requirements Management (2/2)

- **Business** and **technical** environment always changes. New **hardware**, business **priorities** may change, new **legislation** may cause compliance changes
- **Diverse** set of **stakeholders** have differing requirements
- Requirements are a **compromise** and often need to be **re-prioritized**

Requirements Management Functions

- Aims to **understand** and **Control** Requirement Changes
- Manage **Dependent** Requirements
- **Formalizes** the Process of Changes to Requirements and managing the **consequences**



Requirements Management Planning(1/2)

- **Requirements identification** : Each requirement must be **uniquely identified** so that it can be cross-referenced with other requirements and used in **traceability** assessments
- A **change management process** : This is the set of activities that assess the impact and cost of changes
- **Traceability policies** : These policies define the **relationships** between each requirement and between the **requirements** and the **system design** that should be recorded.

Requirements Management Planning(2/2)

Tool support (Excel Sheets or DBMS) is required for...

- **Requirement storage**
Secure and Accessibility Rights
- **Change management**
Tools can keep track of suggested changes and responses to these suggestions.
- **Traceability management**
Tool support for traceability allows related requirements to be discovered and stored.



Requirements Change Management(1/2)

- **Problem analysis and change specification:** The process starts with a change proposal that is analyzed to check if it is **valid**. The change requester then can give feedback with more specific changes or drop the change request.
- **Change analysis and costing:** The cost of making this change is estimated based on the changes required to the requirements document and system design and implementation. Once this analysis is complete, then a decision is made with regards to proceeding with this requirements change.

Requirements Change Management(2/2)

- **Change implementation**

If the requirements change is accepted, then changes are made to the **requirements document, design and implementation**.

A requirements document should be easy to modify without extensive re-writing.

